

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	CSA0620	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Case Study
Weightage:	20% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	A. DEKKSHITHA	Reg. No.:	192572133
Section / Batch:		Date:	06.08.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given case study questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues, provide an in-depth analysis, and propose innovative, feasible solutions with strong justification.

Question Type	Focus Area	Questions
Case Study	Focus on Design Divide and Conquer algorithms: Merge Sort, Quick Sort, Binary Search and Matrix Multiplication	Q1

SECTION A – CASE STUDY**Q1. Digital Music Library Sorting (Merge Sort)**

A digital music platform organizes millions of songs based on release date and popularity. Analyze how Merge Sort can be used to ensure stable and efficient sorting. Identify issues such as large dataset handling and consistency. Apply divide and conquer principles to explain the process. Analyze time complexity and design a solution with justification.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Case Study					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25%	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20%	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20%	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10%	Clear, well structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%				
Application of Concepts	25%				
Analysis	20%				
Solution Design	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. Problem Analysis

A digital music platform stores millions of songs. Users expect songs to be sorted quickly based on release date and popularity. As the number of songs increases, the sorting algorithm must remain efficient and consistent.

Merge Sort is suitable because it uses the Divide and Conquer technique, has $O(n \log n)$ time complexity in all cases, and is a stable sorting algorithm.

2. Key Issues

- Large dataset containing millions of songs.
- Efficient sorting is required to reduce waiting time.
- Songs having the same release date should maintain their original order (stability).
- Performance should remain consistent regardless of the input order.

3. Divide and Conquer Principle

Merge Sort works in three steps:

1. Divide
 - Divide the music library into two equal halves.
2. Conquer
 - Recursively sort each half.
3. Combine
 - Merge the two sorted halves into one sorted list.

4. Algorithm

1. Read the music records.
2. Divide the dataset into two halves.
3. Recursively sort each half.
4. Compare release year.
5. If release years are equal, compare popularity.
6. Merge the sorted lists.
7. Display the sorted songs.
8. Measure execution time.

5. CODE :

```
import random
import time
def generate_music_library(n):
    songs = []
    for i in range(n):
        song = {
            "title": "Song_" + str(i + 1),
            "year": random.randint(2000, 2025),
            "popularity": random.randint(1, 100)
        }
        songs.append(song)
    return songs
def merge(left, right):
    result = []
    i = 0
    j = 0
    while i < len(left) and j < len(right):
        if left[i]["year"] < right[j]["year"]:
            result.append(left[i])
            i += 1
        elif left[i]["year"] > right[j]["year"]:
            result.append(right[j])
            j += 1
        else:
            if left[i]["popularity"] >= right[j]["popularity"]:
                result.append(left[i])
                i += 1
            else:
                result.append(right[j])
                j += 1
    while i < len(left):
        result.append(left[i])
        i += 1
    while j < len(right):
        result.append(right[j])
```

```

        j += 1
    return result
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)
sizes = [1000, 10000, 100000]
print("DIGITAL MUSIC LIBRARY SORTING USING MERGE SORT")
print("=" * 60)
for size in sizes:
    library = generate_music_library(size)
    start = time.perf_counter()
    sorted_library = merge_sort(library)
    end = time.perf_counter()
    print("\nDataset Size :", size)
    print("Execution Time :", round(end - start, 6), "seconds")
    print("\nFirst 5 Sorted Songs")
    for song in sorted_library[:5]:
        print(song)
print("\nTime Complexity : O(n log n)")
print("Space Complexity : O(n)")
print("Merge Sort is Stable and suitable for large datasets.")

```

6. OUTPUT :

DIGITAL MUSIC LIBRARY SORTING USING MERGE SORT

Dataset Size : 1000

Execution Time : 0.002526 seconds

First 5 Sorted Songs

```
{'title': 'Song_946', 'year': 2000, 'popularity': 100}
```

```
{'title': 'Song_96', 'year': 2000, 'popularity': 96}  
{'title': 'Song_682', 'year': 2000, 'popularity': 93}  
{'title': 'Song_142', 'year': 2000, 'popularity': 90}  
{'title': 'Song_289', 'year': 2000, 'popularity': 90}
```

Dataset Size : 10000

Execution Time : 0.037089 seconds

First 5 Sorted Songs

```
{'title': 'Song_769', 'year': 2000, 'popularity': 100}  
{'title': 'Song_5858', 'year': 2000, 'popularity': 100}  
{'title': 'Song_94', 'year': 2000, 'popularity': 99}  
{'title': 'Song_5397', 'year': 2000, 'popularity': 99}  
{'title': 'Song_4676', 'year': 2000, 'popularity': 98}
```

Dataset Size : 100000

Execution Time : 0.579252 seconds

First 5 Sorted Songs

```
{'title': 'Song_12286', 'year': 2000, 'popularity': 100}  
{'title': 'Song_13967', 'year': 2000, 'popularity': 100}  
{'title': 'Song_15361', 'year': 2000, 'popularity': 100}  
{'title': 'Song_16435', 'year': 2000, 'popularity': 100}  
{'title': 'Song_16610', 'year': 2000, 'popularity': 100}
```

Time Complexity : $O(n \log n)$

Space Complexity : $O(n)$

Merge Sort is Stable and suitable for large datasets.

7. EXECUTION :

**SIMATS**
ENGINEERING

SIMATS Online Programming Lab
DAA PROGRAMMING

 Dekkshitha A
192572133

1/1 answered

Your Code

```
1 import random
2 import time
3 def generate_music_library(n):
4     songs = []
5     for i in range (n):
6         song = {
7             "title": "Song_" + str(i+1),
8             "year": random.randint(2000,2025),
9             "popularity": random.randint(1,100)
10        }
11        songs.append(song)
12    return songs
13 def merge(left,right):
14     result = []
15     i=0
16     j=0
17     while i< len(left) and j<len(right):
18         if left[i]["year"]<right[j]["year"]:
19             result.append(left[i])
20             i+=1
```

Input
stdin input

Output

```
2000, 'popularity': 100

Time Complexity : O(n log n)
Space Complexity : O(n)
Merge Sort is Stable and
suitable for large datasets.
```

**SIMATS**
ENGINEERING

SIMATS Online Programming Lab
DAA PROGRAMMING

 Dekkshitha A
192572133

```
18     if left[i]["year"]<right[j]["year"]:
19         result.append(left[i])
20         i+=1
21     elif left[i]["year"]>right[j]["year"]:
22         result.append(right[j])
23         j+=1
24     else:
25         if left[i]["popularity"]>=right[j]["popularity"]:
26             result.append(left[i])
27             i+=1
28         else:
29             result.append(right[j])
30             j+=1
31     while i<len(left):
32         result.append(left[i])
33         i+=1
34     while j< len(right):
35         result.append(right[j])
36         j+=1
37     return result
38 def merge_sort(arr):
39     if len(arr)<=1:
40         return arr
```




SIMATS Online Programming Lab

DAA PROGRAMMING


Dekkshitha A
192572133

```

41 mid = len(arr)// 2
42 left = merge_sort(arr[:mid])
43 right = merge_sort(arr[mid:])
44 return merge(left,right)
45 sizes = [1000,10000,100000]
46 print("DIGITAL MUSIC LIBRARY SORTING USING MERGE SORT")
47 print("="*60)
48 for size in sizes:
49     library = generate_music_library(size)
50     start = time.perf_counter()
51     sorted_library = merge_sort(library)
52     end = time.perf_counter()
53     print("\nDataset Size :",size)
54     print("Execution Time :", round(end - start, 6),"seconds")
55     print("\nFirst 5 Sorted Songs")
56     for song in sorted_library[:5]:
57         print(song)
58 print("\nTime Complexity : O(n log n)")
59 print("Space Complexity : O(n)")
60 print("Merge Sort is Stable and suitable for large datasets.")

```



8. Time Complexity

Case	Complexity
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$
Worst Case	$O(n \log n)$
Space Complexity	$O(n)$

9. Justification

Merge Sort is suitable because:

- Stable sorting algorithm.
- Predictable $O(n \log n)$ performance.
- Handles millions of records efficiently.
- Suitable for external sorting and large databases.
- Divide and Conquer improves scalability.

10. Result

The Digital Music Library was successfully sorted using Merge Sort. The algorithm efficiently handled large datasets while maintaining stable ordering and consistent performance.

11. Conclusion

Merge Sort is an ideal algorithm for digital music platforms because it guarantees **$O(n \log n)$** time complexity in all cases. Its stable nature preserves the order of songs with equal release dates, ensuring consistency. The divide-and-conquer approach makes it highly suitable for processing very large music libraries efficiently.